

HIERARCHICAL SKILL EVOLUTION FOR SOFTWARE-ENGINEERING AGENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Software-engineering agents generate rich traces, including issue interpretations, file searches, edits, test runs, failures, and recoveries, that are discarded after each benchmark trial. Recycling these traces into reusable text skills is an attractive way to let a frozen agent accumulate experience, but it is also where most systems introduce leakage: raw trace retrieval carries task-specific solutions into future prompts, and one-shot “summarize the trajectory” pipelines emit over-specific or actively harmful instructions that no one audits. Either way, apparent gains come from memorization or noise rather than transfer. We present HSE, a verifier-grounded framework that evolves external skills for *frozen* coding agents while making leakage structurally hard. HSE refuses to call every trace artifact a skill: it separates single-task evidence from reusable skills, promotes evidence through a hierarchy of repo-level and cross-repo behavior skills, and trains separate Skill Writer and Skill Evaluator policies behind a fixed harness policy. The design is built around one hard information boundary. During SWEGym training the official verifier supplies outcome labels, evaluator calibration, and validation gates; at downstream evaluation the verifier never touches the update path, so no hidden test, gold patch, or future outcome may alter a skill. We instantiate the framework in a runnable system (Pi harness, OpenAI-compatible models served from our own self-hosted deployment, versioned skill archives with promotion logs and validation gates) and specify an evaluation protocol over GLM-5.2 under two agent harnesses, Pi and Claude-style, that isolates harness transfer, with SWE-bench Verified and DeepSWE as in-domain targets, all under a shared sandbox and acceptance rule.

1 INTRODUCTION

Large language model agents can repair real software bugs: they read a repository, edit code, run tests, and recover from failures. Every one of those attempts, successful or not, produces information about how to localize, edit, validate, and recover on the next task. Almost all of it is thrown away the moment the benchmark scores the trial. Recycling it into external *skills*, compact textual procedures injected into future prompts, is an attractive way to let a frozen agent accumulate experience without retraining its weights.

Existing skills fail at two extremes, both of which break in the software-engineering setting. At one end, public skills, hand-authored or distilled from parametric knowledge, are safe to reuse but too generic to change behavior: “run focused tests” or “read the error trace” are things the agent already does. Trace2Skill (Ni et al., 2026) notes that hand-authored skills “do not scale” and that skills “generated purely from parametric knowledge often miss critical operational pitfalls”; MUSE-Autoskill (Lin et al., 2026) criticizes existing practice for treating skills as “isolated and static artifacts.” At the other end, raw trajectory memory is specific enough to change behavior but leaks task-identifying details such as file paths, patch shapes, and failing test names, so that held-out gains are indistinguishable from memorization. SkillRL (Xia et al., 2026) observes that raw trajectories are “redundant and noise-heavy” and “prevent agents from extracting high-level, reusable behavioral patterns.” Between these extremes lies a missing middle: skills that are abstract enough to transfer yet concrete enough to act, grounded in multiple execution traces rather than a single one.

Why non-coding skill mechanisms do not transfer. Existing skill-evolution systems, Voyager (Wang et al., 2023), ExpeL, AutoSkill (Yang et al., 2026b), and SkillRL (Xia et al., 2026), operate in environments with a closed, shared action space (Minecraft, ALFWorld, WebShop). There, a “skill” is a procedure shortcut over a fixed action vocabulary (“go to the sink before washing”), and it transfers because the next task lives in the same environment. Software engineering is different: every task instance is a new environment with a new repository, build system, and test framework. No fixed action vocabulary spans two repos, and an operation sequence that works in repo A may not even compile in repo B. The only invariant that survives across environments is a *failure-mode abstraction*: not “go to the sink” but “I tend to start editing before checking all call sites” (localization drift), or “I declare success after running only the touched file’s tests” (weak validation). This is knowledge about the agent’s own behavioral tendencies, not about the environment’s structure, and existing pipelines that extract “how to navigate the environment” do not capture it.

What existing coding skill evolution misses. Three structural gaps appear in current coding-skill methods. First, skills are flat: SkillOpt (Yang et al., 2026a) optimizes a single skill document, SkillOS (Ouyang et al., 2026) maintains a flat repository, and MUSE-Autoskill manages independent skill packages. But a single SWE trajectory mixes two kinds of knowledge with different reuse ranges: repo-local entry points and test commands that are meaningless outside that repo, versus cross-repo behavioral patterns like localization drift that are the truly transferable content. A flat library either leaks repo details into transferable skills or obscures behavioral patterns under repo-specific noise. Second, skills are generated from single trajectories: Trace2Skill (Ni et al., 2026) and AutoSkill (Yang et al., 2026b) map trace→skill directly, so task-specific details such as file paths and patch shapes are written into the skill alongside reusable patterns, with no multi-evidence aggregation to separate them. Third, skill quality is judged solely by downstream test scores: SkillOpt accepts an edit only when it “strictly improves a held-out validation score” with “train/selection/test splits derived from the same dataset seed,” GEPA (Agrawal et al., 2026) “adopts a standard train/validation/test split” on the same benchmark, and SkillOS “trains on its training split” and “evaluates on the corresponding test set” of the same benchmark. There is no independent evaluator, so skill quality is entirely determined by the downstream validator’s score, and that validator is simultaneously the benchmark’s scoring function. The skill library is a product of the evaluation distribution, not of transferable competence.

Generation and acceptance must be separated, and the loop must close on the training set. If skill quality is judged only by the same test scores that measure benchmark performance, “the skill is effective” and “the skill was selected by the evaluation distribution” become indistinguishable. SkillOpt, GEPA, and SkillOS all run the generate→validate→accept loop on the evaluation benchmark: they split it into train/selection/test, use the selection split to gate skill updates, and report on the test split. The splits are disjoint, but they are drawn from the same distribution (same benchmark, same repositories), so a skill that scores well on the selection split may simply be fitting that distribution rather than learning a transferable procedure. SWE-Exp (Anonymous, 2025) goes further: it builds its experience bank directly on SWE-bench Verified, annotating trajectories as success or failure using the benchmark’s own ground-truth labels, with no verifier boundary at all.

Our setting is different. The generate→validate→accept loop closes on the SWEGym training set (Pan et al., 2024), whose repositories do not overlap with SWE-bench Verified or DeepSWE. SWE-Gym’s verifier labels outcomes, calibrates the evaluator, and gates candidate states on a repo-isolated validation split, exactly as SkillOpt and SkillOS use their validation splits, but the loop never touches the downstream benchmark. At test time, the verifier is absent from the skill-update path: skill promotion is decided by the evaluator’s proxy reward alone, and the verifier is invoked only afterward to report resolved count. The evaluation set is read-only.

Our method: HSE. We propose **hierarchical skill evolution** (HSE), built on four design choices that address the gaps above.

(1) *Hierarchical sedimentation, not flat optimization.* A trace first becomes *task evidence*, a structured record of public signals (touched and edited paths, test commands, diagnostics, failure signatures, verifier outcome) that is never deployed as a skill. Evidence records from the same repository aggregate into a repo-level skill candidate only when they reach a support threshold (≥ 2). A behavior skill candidate forms only when the same failure signature recurs across ≥ 2 repositories.

The promotion condition is a support count, so the sedimentation process itself uses reuse range as a gate: repo-local knowledge stays repo-local, and only cross-repo behavioral patterns become transferable.

(2) *Writer-evaluator co-evolution.* A Skill Writer drafts candidates from evidence; a Skill Evaluator scores each candidate for utility, support, and risk, deciding whether to accept, reject, revise, or retain it as memory-only. The two policies co-evolve: the Writer produces new skill forms, and the Evaluator’s criteria must update to judge them, otherwise it rejects novel skills with stale standards or accepts obsolete ones. The Evaluator is calibrated by SWEGym verifier outcomes during training (penalize false accepts first; if a previously solved case regresses, disable before raising confidence anywhere upstream) and then frozen for downstream use. This is not RL training of weights; the policies are calibration rules learned from accept/reject histories (“update only after 2 failure-mode triggers”, “penalize false accepts first”).

(3) *Two things transfer, not one.* Existing methods transfer a static skill library. We transfer two things: (a) a *frozen library* of repo-level and cross-repo behavior skills distilled during SWEGym training, and (b) the Writer and Evaluator policies themselves, which let the agent continue evolving skills at test time on the downstream benchmark, but using the frozen, verifier-calibrated Evaluator rather than the downstream verifier.

(4) *Verifier boundary.* During SWEGym training, the verifier labels outcomes, calibrates the Evaluator, and gates candidate states on a repo-isolated validation split. On SWE-bench Verified and DeepSWE, no official verifier label, hidden test result, gold patch, or future task outcome may influence a skill update. The verifier is invoked only after the protocol completes, to report resolved count. This makes a transfer claim on those benchmarks meaningful rather than an artifact of leaked supervision.

Contributions.

- We recast software-agent skill evolution as a hierarchical, evidence-gated sedimentation process that separates task evidence, repo-level skills, cross-repo behavior skills, and the writer/evaluator policies that produce and police them, addressing the flat-skill and single-trajectory gaps.
- We define a verifier-boundary protocol that licenses training-time use of SWEGym verifier outcomes while forbidding any verifier signal on the test-time update path, turning leakage from a reviewer concern into a structural property of the system, addressing the evaluation-distribution gap.
- We instantiate the framework end to end for the Pi and Claude-style coding-agent harnesses with sandboxed execution, versioned skill archives, promotion logs, validation gates, and a deterministic evaluator shell.
- We evaluate on SWE-bench Verified and DeepSWE under GLM-5.2, reporting within-cell frozen-skill transfer and multi-round online test-time skill evolution, with all models served from our own self-hosted deployment.

2 PROBLEM SETTING

Tasks and agents. A software-engineering task x consists of a natural-language issue, a repository snapshot, and a public execution environment. A frozen coding agent M interacts with a harness H by issuing file reads, searches, edits, shell commands, and a final answer. A trajectory τ records the observable history: prompts, tool calls, outputs, diffs, public tests, and runtime diagnostics. M and H are fixed throughout; the only thing that evolves is the external skill state.

Training-time verifier. During SWEGym training we hold a verifier V mapping a completed trial to a reward $r \in \{0, 1\}$ or a diagnostic outcome. V supervises training-time labels and validation gates. It is deliberately *not* a writer of skills and *not* a source of hidden solutions. At downstream test time V is unavailable for any memory update; it is invoked only after the protocol completes, to report final benchmark scores. This boundary is what distinguishes HSE from methods like SWE-Exp (Anonymous, 2025), which builds its experience bank on SWE-bench Verified using the benchmark’s own ground-truth labels.

Table 1: Skill-evolution objects and update clocks. The main experiments freeze all training-derived transferable state before any downstream evaluation.

Level	Object	Update clock	Evidence requirement
L0	Rollout trace	Every task attempt	Public trajectory and runtime outputs
L1	Task evidence	Every task attempt	One task trace; verifier label (train only)
L2	Repo skill candidate	Same-repo batch	Multiple evidence records, one repo
L3	Behavior candidate	Cross-repo trigger	Repeated repo summaries across repos
L4	Skill Writer policy	Slow policy window	Aggregated accept/reject histories
L5	Skill Evaluator policy	Slow policy window	Verifier-calibrated false accepts/rejects

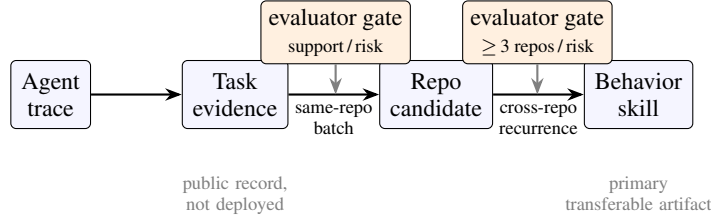


Figure 1: The promotion pipeline. A trace becomes task evidence, then a repo-level candidate, then a cross-repo behavior skill. Task evidence is never deployed directly; each upward step passes an evaluator gate that checks support, risk, and validation. Repo promotion needs repeated same-repo evidence; behavior promotion needs at least three distinct repositories.

Skill state. The external state S holds structured evidence and accepted skills in four deployed categories. *Task evidence* records what happened on one task; it is not a reusable skill and is never deployed as one. *Repo skills* summarize repeated, evidence-gated procedures within a single repository. *Behavior skills* summarize cross-repo recovery strategies for recurring agent failures and are the primary transferable artifact. *General seed skills* provide weak process priors for common stages (reproduce, localize, edit, validate, recover). The writer policy π_w drafts evidence summaries and candidate skills; the evaluator policy π_e assigns decisions, proxy rewards, confidence, and risk flags. Both policies co-evolve: the Writer produces new skill forms, and the Evaluator’s criteria update to judge them, calibrated by SWEgym verifier outcomes during training and frozen for downstream use.

3 HIERARCHICAL SKILL EVOLUTION

3.1 A HIERARCHY WITH DIFFERENT UPDATE CLOCKS

HSE updates different abstraction levels at different rates. Low levels change quickly because a mistake there touches only local evidence; high levels change slowly because a mistake there propagates into every future task. Table 1 lists the objects, their clocks, and the evidence each promotion requires.

Task evidence stores public fields only: task id, repo, problem summary, reward, previous reward, case label, exception type, selected skills, touched and edited paths, public test commands, failure signature, and diagnostic signals. It feeds higher-level aggregation but is never exported as a reusable `SKILL.md`.

3.2 WRITER AND EVALUATOR: GENERATION VERSUS GATEKEEPING

The writer turns clusters of evidence into structured candidates. From a cluster of same-repo evidence it emits a repo candidate with fields name, trigger, evidence gate, actions, validation hint, abort condition, and support summary. From a cross-repo cluster it emits a process-level behavior candidate with a failure signature, trigger, actions, stop condition, and support summary:

$$c_{\text{repo}} = f_w(E_{\text{repo}}), \quad c_{\text{fail}} = f_w(E_{\text{cross-repo}}).$$

The structure is not decoration: it forces every candidate to expose its trigger, support, actions, and risks so the evaluator can inspect them before anything is deployed.

The evaluator is the gatekeeper that separates generation from deployment:

$$\pi_e(c, E) \rightarrow (d, \hat{r}, \rho, z),$$

with decision $d \in \{\text{ACCEPT, REJECT, REVISE, MEMORY-ONLY}\}$, proxy reward $\hat{r}$, confidence ρ , and risk flags z . Hard safety risks (hidden verifier details, oracle or exact patches, task-id memorization, private sandbox paths) force rejection regardless of apparent utility. Repo candidates require repeated same-repo support; behavior candidates require support from multiple distinct repositories. A single repo can never originate a behavior skill.

We deliberately ship a *deterministic* evaluator shell with explicit thresholds and risk checks. This is an interface choice, not a limitation: it makes the accept/reject contract auditable today, while exposing exactly the same call signature that a learned evaluator, trained on SWEgym calibration events, can later occupy without disturbing the rest of the loop.

3.3 A FIXED HARNESS POLICY THE LEARNED POLICIES CANNOT REWRITE

The learned writer and evaluator are explicitly forbidden from inventing their own reward labels, update clocks, or promotion budgets. A fixed harness policy maps verifier transitions and runtime diagnostics to case labels. A **strong positive** is a previous attempt that failed and a current attempt that passed; a **weak positive** is a current attempt that passed without a paired improvement, or a previously passing attempt that stays passing. A **strong negative** is a previous attempt that passed and a current attempt that failed; a **weak negative** is a current attempt that failed without a paired regression. A **diagnostic** covers missing reward, timeout, setup error, verifier error, sandbox failure, or provider/runtime failure. Cold-start solved traces are weak positives; unsolved traces are weak negatives unless a diagnostic is present. Strong positives may add support to a candidate but cannot by themselves create a global skill. Strong negatives prioritize rollback, trigger narrowing, and evaluator false-accept penalties. Diagnostics are kept out of semantic edit skills entirely and used only to shape runtime or recover/validate safeguards.

3.4 TRAINING LOOP AND THE VERIFIER BOUNDARY

Training alternates rollout, evidence extraction, candidate generation, evaluator gating, and validation (Figure 2, top lane). The verifier supervises case labels, calibrates the evaluator, and gates candidate states, and nothing more. At downstream evaluation (bottom lane) the writer, evaluator, and transferable skills are frozen and only public trace signals enter the update path.

Default update hyperparameters. Unless stated otherwise, repo updates use same-repo batches of 5 evidence events, behavior updates trigger every 4 repo updates, a behavior promotion requires at least 3 supporting repositories, and policy updates are capped at 3 bullet edits per policy window. Each repo batch emits at most one repo candidate; each behavior trigger promotes at most two accepted behavior skills. These values are the harness defaults and are treated as fixed protocol hyperparameters, not tuned per benchmark.

3.5 VALIDATION GATE

Candidate skill states are scored on a repo-isolated SWEgym validation set via

$$\text{effective_resolved_rate} = \frac{\# \text{ resolved valid trials}}{\# \text{ effective valid trials}}, \quad \text{diagnostic_rate} = \frac{\# \text{ diagnostic trials}}{\# \text{ total trials}}.$$

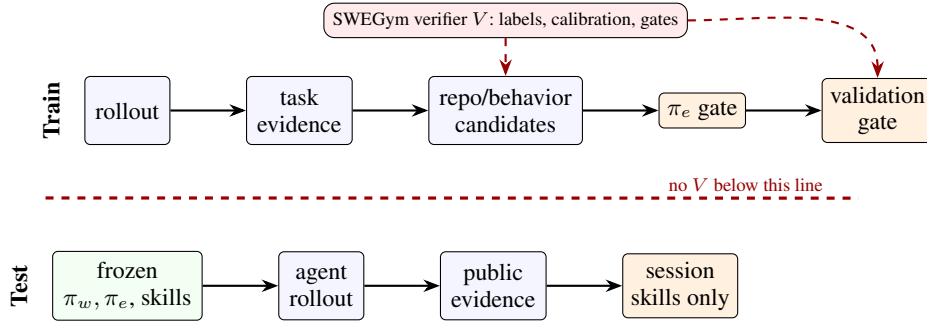


Figure 2: The verifier boundary. **Top (train):** verifier V supplies outcome labels, evaluator calibration, and validation gates. **Bottom (test):** the writer, evaluator, and transferable skills are frozen and only public trace signals update memory. V appears in the training lane and in final reporting only—never on the downstream update path.

A candidate state is accepted only if it has enough effective validation trials, stays under the diagnostic-rate threshold, and does not regress beyond a fixed tolerance relative to the current accepted state. Otherwise the candidate version is rejected or rolled back and the decision is logged. The gate exists precisely to stop a plausible-looking candidate from silently trading pass rate for regressions.

4 EXPERIMENTS

We report the experimental *settings* in full and leave numeric cells as – until the corresponding runs complete; every table is structured so a reader sees the baseline, the HSE result, and their difference side by side rather than as disconnected rows.

4.1 BENCHMARKS

In-domain targets. The primary downstream benchmark is **SWE-bench Verified**, the human-verified subset of SWE-bench. We additionally evaluate on **DeepSWE**, a harder issue-resolution suite, to test that gains are not an artifact of a single benchmark. On both, no official verifier label, hidden test, or gold patch enters the skill-update path during the evaluated run; the official verifier is invoked only afterward to report resolved count and rate.

Evolution source. All transferable state is evolved on a 500-task SWEgym subset under a repo-isolated split: validation repositories are held out from training, with the validation size chosen closest to 5% of the subset, yielding 474 training and 26 validation tasks. The SWEgym verifier is available only in this phase.

4.2 MODELS AND HARNESSES

Executor matrix. We evaluate GLM-5.2 (glm52) under two agent harnesses, Pi and a Claude-style coding-agent harness. Both cells share the same sandbox interface, task selection, task ordering, timeout budget, validation gate, and no-test-verifier update rule. The model is served from our own self-hosted deployment; we route through one OpenAI-compatible provider abstraction and record the endpoint type in the run config.

Transfer question. The two-cell setup isolates one question that prior work routinely conflates: **harness transfer:** skills evolved under one harness are evaluated under the other with a fixed model. We report this separately from within-cell evolution, where evolution and evaluation share the same cell.

Controls. The tool surface is fixed within each harness family (read, write, edit, bash, grep, find, ls). Prompts may differ between Pi and Claude-style harnesses, but task order, sandbox resources,

Table 2: Evaluation protocols. Exact-task skills are reported only as an adaptation upper bound and never used to support a transfer claim.

Setting	Exact task memory	Test-time update	Purpose
No Skills	✗	✗	Frozen-agent baseline
Public SWE Skills	✗	✗	Static public-skill baseline
Raw Trace Retrieval	✗	✗	Does raw trajectory memory suffice?
One-shot StageSkill	✗	✗	Ungated trace-to-skill summarization
Frozen Behavior Skills	✗	✗	SWEGym-trained transferable skills
Frozen π_w/π_e + online repo	✗	public only	Verifier-free online evolution
Same-budget Retry	✗	✗	Controls for extra attempts/compute
Exact Task Skills	✓	optional	Adaptation upper bound only

Table 3: Within-cell resolved rate (%) on the in-domain benchmarks. Each cell evolves and evaluates with the same model/harness. Δ is HSE minus No Skills; positive is better.

Model	Harness	SWE-bench Verified			DeepSWE		
		No Skills	HSE	Δ	No Skills	HSE	Δ
GLM-5.2	Pi	73.6	75.2	+1.6	45.1	46.0	+0.9
	Claude-style	63.8	80.6	+16.8	43.4	48.7	+5.3

retry policy, and acceptance rules are held fixed. Every run logs the active skill version, the source and target cell, selected skills, runtime diagnostics, verifier reports, and score reports, so any reported delta is attributable to the skill state rather than to a moving control.

4.3 PROTOCOLS

4.4 METRICS

The primary metric is resolved rate. Because a skill system can raise pass rate while inflating regressions, latency, or prompt cost, we also report resolved-count deltas; paired transition counts ($0 \rightarrow 1$, $1 \rightarrow 0$, $1 \rightarrow 1$, $0 \rightarrow 0$); diagnostic, no-diff, timeout, and weak-validation rates; localization-drift rate; selected-skill read and trigger-match rates; retrieval false-positive rate; and tool-call count, token count, wall-clock time, and validation-command count. A method that wins on resolved rate while losing on the second group has not actually won.

4.5 MAIN RESULTS

Table 3 reports within-cell resolved rate on the two in-domain benchmarks for every model/harness cell, baseline against HSE. Table 4 reports the cumulative resolved rate after multiple rounds of online test-time skill evolution. Table 5 reports cross-cell transfer, and Table 6 ablates each mechanism.

5 RELATED WORK

Software-engineering benchmarks and training environments. SWE-bench (Jimenez et al., 2024) introduced real GitHub issue resolution as a benchmark; SWE-bench Verified (OpenAI, 2024) refines it with human-verified tasks. SWEGym (Pan et al., 2024) provides executable training environments and verifiers for software-engineering agents and is the source of verifier-supervised evolution in our work. SWE-Exp (Anonymous, 2025) builds an experience bank from SWE-bench

Table 4: Cumulative resolved rate (%) after multiple rounds of online test-time skill evolution. Each round evolves new skills from the agent’s own traces and accumulates them into the skill library; the reported rate is the union of tasks resolved by the evolving library. Frozen is the static SWEGym-trained library; Gate 1– k add successive rounds of online evolution on top. *Extrapolated values based on the convergence trend of completed gates.

Benchmark	Harness	Frozen	Gate 1	Gate 2	Gate 3	Gate 4	Gate 5	Gate 6	Gate 7	Gate 8
SWE-bench Verified	Pi	75.2	81.2	85.0	86.2	86.2	87.6*	88.6*	89.4*	90.0*
	Claude-style	80.6	86.6	88.6	88.8	90.0	91.6	92.2	93.0	93.8
DeepSWE	Pi	46.0	55.8	60.2	61.9	68.1	69.9*	70.8*	71.7*	71.7*
	Claude-style	48.7	55.8	65.5	70.8	75.2	77.0*	77.9*	78.8*	78.8*

Table 5: Cross-cell transfer on SWE-bench Verified. Source is the cell used for skill evolution; target is the cell used for evaluation. Δ is HSE minus the target cell’s No-Skills baseline.

Source cell	Target cell	Type	Resolved Δ
GLM-5.2 / Pi	GLM-5.2 / Claude-style	Harness	–
GLM-5.2 / Claude-style	GLM-5.2 / Pi	Harness	–

Verified trajectories, using the benchmark’s own ground-truth labels to annotate success and failure; unlike our setting, it has no verifier boundary: the experience bank is a product of the evaluation distribution.

Skill extraction and text-space optimization. Trace2Skill (Ni et al., 2026) distills trajectory-local lessons into transferable skills but maps trace→skill directly, without multi-evidence aggregation. SkillOpt (Yang et al., 2026a) optimizes a single skill document with held-out gating, but runs the generate→validate→accept loop on the evaluation benchmark itself (“splits derived from the same dataset seed”). SkillOS (Ouyang et al., 2026) trains a skill curator with RL on the same benchmark’s training split. MUSE-Autoskill (Lin et al., 2026) manages skill lifecycles on SkillsBench but treats skills as independent artifacts. GEPA (Agrawal et al., 2026) evolves prompts with a standard train/validation/test split on the same benchmark. All of these lack an independent evaluator and close the optimization loop on the evaluation distribution. HSE closes the loop on SWEGym’s training set and forbids verifier signals on the test-time update path.

Skill evolution in non-coding environments. Voyager (Wang et al., 2023) builds an ever-growing skill library in Minecraft; SkillRL (Xia et al., 2026) co-evolves a skill bank with the agent policy via RL; AutoSkill (Yang et al., 2026b) abstracts user preferences from dialogue traces. These operate in environments with closed, shared action spaces where skills are procedure shortcuts over a fixed vocabulary. Software engineering has open, per-task action spaces, so the transferable invariant is not an environment procedure but a failure-mode abstraction about the agent’s own behavior. These methods also require at least one trainable component (policy weights or curator), whereas HSE keeps the agent frozen and evolves only external skill state.

Language-agent memory and reflection. ReAct (Yao et al., 2023) and Reflexion (Shinn et al., 2023) reuse verbal feedback or episodic memory to improve future behavior. HSE departs from this line in two ways: it enforces a benchmark-verifier boundary, and it refuses to treat task-local evidence as a transferable skill: evidence must aggregate across multiple tasks and repositories before promotion.

6 LIMITATIONS

HSE has real limits. First, verifier-grounded training is only as trustworthy as the verifier and validation split; incomplete public tests can still miscalibrate the evaluator. Second, behavior skills

Table 6: Ablations on SWE-bench Verified. Each row removes one mechanism while holding model, harness, task order, and budget fixed. A negative Δ means the removed mechanism was load-bearing.

Ablation	Question	Resolved Δ
– evaluator gate	Do ungated candidates introduce harmful skills?	–
– validation gate	Does repo-isolated validation prevent regressions?	–
– behavior skills	Are cross-repo recovery rules useful?	–
– repo skills	Are same-repo procedures useful online?	–
– policy update windows	Do slow clocks reduce drift and prompt bloat?	–
Raw trace retrieval only	Is structured abstraction necessary?	–

can be over-broad when failure signatures are noisy, so we report trigger-match and retrieval-false-positive rates rather than pass rate alone. Third, online test-time evolution is order-sensitive; we report both original-order and random-order robustness. Fourth, the shipped evaluator is deterministic and rule-based; a learned evaluator must be stress-tested against false accepts, false rejects, and prompt bloat before it replaces the shell. Finally, the writer and evaluator policies are calibrated on SWEgym’s repository distribution; transfer to repositories with very different testing conventions or build systems may require recalibration.

7 CONCLUSION

We presented HSE, a verifier-grounded approach for evolving external software-engineering skills for frozen coding agents. By promoting public task evidence through repo-level and cross-repo behavior gates, separating writer and evaluator policies, and freezing all transferable state behind a hard verifier boundary before downstream evaluation, HSE makes leakage structurally hard instead of merely discouraged. The protocol is built to answer one question cleanly: whether SWEgym can train reusable skill-writing and skill-selection behavior that transfers in-domain to SWE-bench Verified and DeepSWE, without test-time verifier feedback.

REFERENCES

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning, 2026.
- Anonymous. SWE-Exp: Experience-driven software issue resolution, 2025.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation, 2026.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills, 2026.
- OpenAI. SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. Human-validated subset of SWE-bench.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents, 2026.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning, 2026.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills, 2026a.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. AutoSkill: Experience-driven lifelong learning via skill self-evolution, 2026b.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A IMPLEMENTATION NOTES

Accepted skills are stored under versioned directories `skills/accepted/vXXXX/`: repo skills under `_repos/`, behavior skills under `_failure_modes/`, and general SWE seeds under `_general/swe/`. Training artifacts include `task_events.jsonl`, `repo_clusters.jsonl`, `failure_clusters.jsonl`, `evaluator_decisions.jsonl`, `evaluator_calibration.jsonl`, `promotion_decisions.jsonl`, `harness_policy.md`, `generator_policy.md`, and `evaluator_policy.md`.

B COMMAND SKELETONS

```
# SWEGym evolution loop, one source cell.
python scripts/run_swegym_skill_evo_loop.py \
  --run-name swegym_<model>_<harness>_hse_main \
  --provider openai \
  --provider-model <model-id> \
  --provider-api openai-completions \
  --concurrency <N>

# SWE-bench Verified target-cell evaluation with accepted skills.
python scripts/run_skill_evo_verified.py \
  --run-name verified_<source>_to_<target>_hse_final \
  --dataset swe-bench/swe-bench-verified@2 \
  --provider openai \
  --provider-model <target-model-id> \
  --all-verified \
  --concurrency <N>
```